DATABASE RELATIONSHIP ANALYSIS USING JOINS

Introduction

Database Joins are used to combine data from two or more tables based on a common column. In the Farm2Home E-Commerce Database, joins help retrieve customer details, product information, order details, and payment information from different tables.

This experiment uses four tables: CUSTOMER, PRODUCT, ORDERS, and PAYMENT.

1. Combine Customer, Product, Order, and Payment Tables

The JOIN operation combines records from multiple tables using related columns.

In Farm2Home:

- CUSTOMER stores customer information.
- PRODUCT stores agricultural product details.
- ORDERS stores customer order information.
- PAYMENT stores payment details.

```
SQL> SELECT
  2      C.Customer_ID,
  3      C.Customer_Name,
  4      P.Product_ID,
  5      P.Product_Name,
  6      O.Order_ID,
  7      O.Quantity,
  8      PAY.Payment_ID,
  9      PAY.Amount,
 10      PAY.Payment_Mode,
 11      PAY.Payment_Status
 12  FROM CUSTOMER C
 13  JOIN ORDERS O
 14  ON C.Customer_ID = O.Customer_ID
 15  JOIN PRODUCT P
 16  ON O.Product_ID = P.Product_ID
 17  JOIN PAYMENT PAY
 18  ON O.Order_ID = PAY.Order_ID;
ON C.Customer_ID = O.Customer_ID
```

2. Implement INNER JOIN, LEFT JOIN, and RIGHT JOIN

2.1 INNER JOIN

INNER JOIN returns only the matching records from both tables.

SQL Query:

SELECT C.Customer_Name, O.Order_ID,

P.Product_Name, O.Quantity

FROM CUSTOMER C

INNER JOIN ORDERS O

ON C.Customer_ID = O.Customer_ID

INNER JOIN PRODUCT P

ON O.Product_ID = P.Product_ID;

Advantages:

- Retrieves matching records.
- Combines related table information.
- Avoids unrelated records.

```
SQL> SELECT C.Customer_ID,
  2         C.Customer_Name,
  3         O.Order_ID,
  4         P.Product_Name,
  5         O.Quantity
  6  FROM CUSTOMER C
  7  INNER JOIN ORDERS O
  8  ON C.Customer_ID = O.Customer_ID
  9  INNER JOIN PRODUCT P
 10  ON O.Product_ID = P.Product_ID;
ON C.Customer_ID = O.Customer_ID
```

2.2 LEFT JOIN

LEFT JOIN returns all records from the left table and matching records from the right table. If there is no match, NULL values are displayed.

SQL Query:

SELECT C.Customer_Name,

O.Order_ID, O.Order_Date

FROM CUSTOMER C

LEFT JOIN ORDERS O

ON C.Customer_ID = O.Customer_ID;

Advantages:

- Displays all customers.
- Identifies customers without orders.
- Helps analyze customer activity.

```
SQL> SELECT C.Customer_ID,
  2         C.Customer_Name,
  3         O.Order_ID,
  4         O.Order_Date
  5  FROM CUSTOMER C
  6  LEFT JOIN ORDERS O
  7  ON C.Customer_ID = O.Customer_ID;
ON C.Customer_ID = O.Customer_ID
```

2.3 RIGHT JOIN

RIGHT JOIN returns all records from the right table and matching records from the left table.

SQL Query:

```
SELECT C.Customer_Name,
    O.Order_ID, O.Quantity
FROM CUSTOMER C
RIGHT JOIN ORDERS O
ON C.Customer_ID = O.Customer_ID;
```

Advantages:

- Displays all orders.
- Retrieves matching customer details.
- Helps analyze order information.

```
SQL> SELECT C.Customer_ID,
  2         C.Customer_Name,
  3         O.Order_ID,
  4         O.Quantity
  5  FROM CUSTOMER C
  6  RIGHT JOIN ORDERS O
  7  ON C.Customer_ID = O.Customer_ID;
```

3. Retrieve Complete Order Details

This query retrieves complete order information, including customer name, product name, quantity, price, total amount, and payment details.

```
SQL> SELECT
  2      C.Customer_ID,
  3      C.Customer_Name,
  4      O.Order_ID,
  5      P.Product_Name,
  6      O.Quantity,
  7      P.Price,
  8      (O.Quantity * P.Price) AS Total_Amount,
  9      PAY.Payment_Mode,
 10      PAY.Payment_Status
 11  FROM CUSTOMER C
 12  JOIN ORDERS O
 13  ON C.Customer_ID = O.Customer_ID
 14  JOIN PRODUCT P
 15  ON O.Product_ID = P.Product_ID
 16  LEFT JOIN PAYMENT PAY
 17  ON O.Order_ID = PAY.Order_ID;
```

Advantages:

- Displays complete order information.
- Calculates the total product amount.
- Shows payment mode and status.
- Helps track customer orders.

4. Display Customer Purchase History

Customer purchase history displays the products purchased by customers along with quantity and order date.

```
SQL> SELECT
  2      C.Customer_ID,
  3      C.Customer_Name,
  4      O.Order_ID,
  5      P.Product_Name,
  6      O.Quantity,
  7      O.Order_Date
  8  FROM CUSTOMER C
  9  JOIN ORDERS O
 10  ON C.Customer_ID = O.Customer_ID
 11  JOIN PRODUCT P
 12  ON O.Product_ID = P.Product_ID
 13  ORDER BY C.Customer_Name;
```

Advantages:

- Displays customer purchase details.
- Tracks previously ordered products.

- Helps understand customer buying patterns.
- Supports better customer service.

5. Generate Multi-Table Reports

Multi-table reports combine information from multiple tables to generate useful business reports.

```
SQL> SELECT
  2      O.Order_ID,
  3      C.Customer_Name,
  4      P.Product_Name,
  5      O.Quantity,
  6      P.Price,
  7      (P.Price * O.Quantity) AS Total_Sales,
  8      PAY.Payment_Mode,
  9      PAY.Payment_Status
 10  FROM CUSTOMER C
 11  JOIN ORDERS O
 12  ON C.Customer_ID = O.Customer_ID
 13  JOIN PRODUCT P
 14  ON O.Product_ID = P.Product_ID
 15  LEFT JOIN PAYMENT PAY
 16  ON O.Order_ID = PAY.Order_ID
 17  ORDER BY O.Order_ID;
```

Advantages:

- Generates complete sales reports.
- Displays customer and product details.
- Tracks payment information.
- Helps administrators monitor business transactions.

Conclusion

Thus, Database Relationship Analysis using Joins is implemented successfully in the Farm2Home E-Commerce Database. INNER JOIN, LEFT JOIN, and RIGHT JOIN are used to combine related records from multiple tables. These operations help retrieve complete order details, display customer purchase history, and generate multi-table reports efficiently.